

Epsilon maszynowy (macheps)

Epsilon maszynowy, znany również jako macheps, to wielkość stosowana w informatyce i matematyce do określenia precyzji reprezentacji liczb zmiennoprzecinkowych w komputerach. Jest to najmniejsza wartość, która może być dodana do liczby zmiennoprzecinkowej, aby uzyskać liczbę różną od niej samej, biorąc pod uwagę ograniczenia precyzji w danym systemie numerycznym.

Matematycznie, epsilon maszynowy to najmniejsza liczba $\text{macheps} > 0$, taka że:

$$\text{fl}(1.0 + \text{macheps}) > 1.0$$

oraz

$$\text{fl}(1.0 + \text{macheps}) = 1.0 + \text{macheps}$$

gdzie fl oznacza funkcję zaokrąglającą do najbliższej liczby reprezentowanej w danym systemie numerycznym (floating-point). Innymi słowy, macheps jest odległością od kolejnej liczby maszynowej.

Eksperymentalne wyznaczenie epsilon maszynowego

W celu dokładniejszego zrozumienia precyzji obliczeń zmiennoprzecinkowych w komputerach, przeprowadzone zostało eksperymentalne wyznaczenie epsilon maszynowego dla liczb zmiennoprzecinkowych różnych precyzji: Float16, Float32 i Float64. Zostało ono zrealizowane poprzez iteracyjne mnożenie początkowej wartości przez 0.5, aż do momentu, gdy dodanie tej wartości do liczby 1.0 przestanie powodować różnicę w reprezentacji liczbowej (czyli gdy $1.0 + \text{macheps}$ pozostanie równe 1.0). W celu weryfikacji poprawności algorytmu, uzyskane wyniki porównano z wartościami wyznaczonymi przez funkcję `eps()` z języka Julia, która oblicza teoretyczną wartość *macheps* dla danego typu liczby zmiennoprzecinkowej oraz wartościami `FLT_EPS` oraz `DBL_EPS` zdefiniowanymi w pliku `<float.h>`.

```
In [5]: function compute_macheps(type)
        macheps = type(1)
        while type(1) + type(macheps * type(0.5)) != type(1)
            macheps *= type(0.5)
        end
        return macheps
    end
```

```
Out[5]: compute_macheps (generic function with 1 method)
```

```
In [24]: using PrettyTables

macheps_float16 = compute_macheps(Float16)
macheps_float32 = compute_macheps(Float32)
```

```

macheps_float64 = compute_macheps(Float64)

data = ["Float16" "$macheps_float16" "$(eps(Float16))" "-";
        "Float32" "$macheps_float32" "$(eps(Float32))" "1.192093e-07";
        "Float64" "$macheps_float64" "$(eps(Float64))" "2.220446049250313e-16"]

pretty_table(data, header=["Type", "Machine epsilon", "Machine epsilon (Julia)", "Machine epsilon (C)"],
              alignment=:l, header_crayon = crayon"green")

```

Type	Machine epsilon	Machine epsilon (Julia)	Machine epsilon (C)
Float16	0.000977	0.000977	-
Float32	1.1920929e-7	1.1920929e-7	1.192093e-07
Float64	2.220446049250313e-16	2.220446049250313e-16	2.220446049250313e-16

Dokładność wyznaczania epsilonu maszynowego

Zgodność wyników obliczonych eksperymentalnie z wartościami zwróconymi przez funkcję `eps()` wskazuje na poprawność działania algorytmu i adekwatność zastosowanego kryterium `while type(1) + type(macheps * type(0.5)) != type(1)`.

Algorytm skutecznie wykrywa moment, gdy dodanie danej wartości do 1.0 nie powoduje już zmiany wartości.

Arytmetyka zmiennoprzecinkowa a wartość epsilonu maszynowego

Mniejsza wartość epsilon maszynowy oznacza większą precyzję obliczeń, co jest kluczowe w arytmetyce. W obliczeniach numerycznych zawsze występują błędy zaokrągleń, a liczba `macheps` informuje, jak niewielkie mogą być te błędy. Podczas porównywania dwóch liczb zmiennoprzecinkowych w programach, epsilon maszynowy pomaga ustalić, czy różnica między nimi jest na tyle istotna, aby uznać je za różne.

Epsilon Kahana

Alternatywny sposób liczenia epsilonu maszynowego zaproponował William Kahan. Kahan zwrócił uwagę, że epsilon maszynowy można obliczyć poprzez prostą operację:

$$3(4/3 - 1) - 1$$

Uzasadnienie poprawności wzoru dla arytmetyki Float64

W arytmetyce `Float64` epsilon maszynowym jest liczba $\text{macheps} = 2^{-52}$. W arytmetyce tej mamy zatem:

$$4 = 2^{54} \cdot \text{macheps}$$

Przy wykonywaniu operacji $\frac{4}{3}$ dzielimy machepsy na trzy równe części. Zauważmy, że:

$$2^{54} \bmod 3 = 1$$

co oznacza, że $2^{54} - 1$ machepsów rozdzielimy na trzy grupy, a pozostała $\frac{1}{3}$ machepsa zostanie wliczona w błąd obliczeniowy. W rezultacie otrzymamy liczbę o $\frac{1}{3}$ machepsa za małą.

W wyniku późniejszego mnożenia przez 3 błąd obliczeniowy będzie równy całemu machepsowi. Z tego właśnie powodu:

$$\text{kahan_eps} - \text{eps} = -\text{macheps}$$

Uzasadnienie poprawności wzoru dla arytmetyki Float32

W arytmetyce `Float32` epsilon maszynowym jest liczba $\text{macheps} = 2^{-23}$. Oznacza to, że w przypadku dzielenia przez 3 otrzymujemy wartość, która jest o $\frac{1}{3}$ machepsa za dużą z powodu zaokrąglenia w górę. W rezultacie skutkuje to tym, że otrzymana liczba jest o całego machepsa większa niż oczekiwana wartość.

Z tego powodu, w arytmetyce `Float32`, można sformułować następujące równanie dotyczące błędu obliczeniowego:

$$\text{kahan_eps} - \text{eps} = \text{macheps}$$

Eksperymentalne wyznaczenie wartości epsilonu Kahana

W celu poparcia powyższych dowodów, został zaimplementowany algorytm liczący epsilon Kahana dla `Float16`, `Float32` oraz `Float64`.

```
In [45]: function kahan_eps(type)
          return type(3)*(type(4)/type(3) - type(1)) - type(1)
        end
```

```
Out[45]: kahan_eps (generic function with 1 method)
```

```
In [44]: using PrettyTables

kahan_eps_16 = kahan_eps(Float16)
```

```

kahan_eps_32 = kahan_eps(Float32)
kahan_eps_64 = kahan_eps(Float64)

data = ["Float16" "$kahan_eps_16" "$(abs(kahan_eps_16))" "$(eps(Float16))"
        "Float32" "$kahan_eps_32" "$(abs(kahan_eps_32))" "$(eps(Float32))"
        "Float64" "$kahan_eps_64" "$(abs(kahan_eps_64))" "$(eps(Float64))"

pretty_table(data, header=["Type", "Kahan's epsilon", "Kahan's epsilon (abs)", "Machine epsilon"],
              alignment=:l, header_crayon = crayon"green")

```

Type	Kahan's epsilon	Kahan's epsilon (abs)	Machine epsilon
Float16	-0.000977	0.000977	0.000977
Float32	1.1920929e-7	1.1920929e-7	1.1920929e-7
Float64	-2.220446049250313e-16	2.220446049250313e-16	2.220446049250313e-16

Zgodnie z przewidywaniami, metoda liczenia epsilonu maszynowego zaproponowana przez Williama Kahana okazała się być poprawna co do wartości bezwzględnej.

Liczby zdenormalizowane w standardzie IEEE 754

Standard IEEE 754 wprowadza mechanizm zdenormalizowanych liczb, który pozwala na reprezentację mniejszych wartości ułamkowych, które nie mogłyby być przedstawione w formie znormalizowanej. W przypadku takich liczb:

- Wiodący bit mantysy jest ustawiony na 0.
- Cecha (ekspozycja) ma wszystkie bity równe 0.

Wzór dla zdenormalizowanej liczby w precyzji pojedynczej (single precision)

Zdenormalizowana liczba w formacie pojedynczej precyzji jest opisana wzorem:

$$(-1)^s \times 0.b_1b_2b_3 \dots b_{23} \times 2^{-126}$$

gdzie:

- s to bit znaku (0 dla liczby dodatniej, 1 dla liczby ujemnej),
- $b_1, b_2, \dots, b_{23}$ to bity mantysy.

Wzór dla zdenormalizowanej liczby w precyzji podwójnej (double precision)

Zdenormalizowana liczba w formacie podwójnej precyzji jest opisana wzorem:

$$(-1)^s \times 0.b_1b_2b_3\dots b_{52} \times 2^{-1022}$$

gdzie: s to bit znaku,

- $b_1, b_2, \dots, b_{52}$ to bity mantysy.

Eksperymentalne wyznaczenie najmniejszej liczby zdenormalizowanej

W celu zrozumienia problemu liczb zdenormalizowanych w obliczeniach zmiennoprzecinkowych, przeprowadzono eksperymentalne wyznaczenie wartości najmniejszej liczby zdenormalizowanej `eta` dla typów `Float16`, `Float32` i `Float64`. Proces polegał na iteracyjnym mnożeniu początkowej wartości 1.0 przez 0.5, aż do momentu, gdy uzyskana wartość przestanie być większa od zera.

Aby zweryfikować poprawność algorytmu, wyniki porównano z wartościami uzyskanymi przy użyciu funkcji `nextfloat(x)` z języka Julia. Funkcja ta zwraca najmniejszą liczbę w danej arytmetyce po wartości x .

```
In [7]: function compute_eta(type)
        eta = type(1)
        while type(eta * type(0.5)) > type(0)
            eta *= type(0.5)
        end
        return eta
    end
```

Out[7]: compute_eta (generic function with 1 method)

```
In [8]: using PrettyTables

eta_16 = compute_eta(Float16)
eta_32 = compute_eta(Float32)
eta_64 = compute_eta(Float64)

data = ["Float16" "$eta_16" "$(nextfloat(Float16(0)))";
        "Float32" "$eta_32" "$(nextfloat(Float32(0)))";
        "Float64" "$eta_64" "$(nextfloat(Float64(0)))"]

pretty_table(data, header=["Type", "Eta", "Next float"], alignment=:l, he
```

Type	Eta	Next float
Float16	6.0e-8	6.0e-8
Float32	1.0e-45	1.0e-45
Float64	5.0e-324	5.0e-324

Dokładność wyznaczania najmniejszej liczby zdenormalizowanej

Zgodność wyników obliczonych eksperymentalnie z wartościami zwróconymi przez funkcję `nextfloat()` wskazuje na poprawność działania algorytmu oraz adekwatność zastosowanego kryterium w pętli.

Arytmetyka zmiennoprzecinkowa a liczby zdenormalizowane

Liczby zdenormalizowane pozwalają zwiększyć precyzję obliczeń w arytymetyce zmiennoprzecinkowej. Umożliwiają one kontynuację obliczeń w sytuacji, gdy wartości spadają poniżej granic reprezentacji znormalizowanej. Aby lepiej zobrazować skalę tego problemu, przyjrzyjmy się porównaniu najmniejszych wartości znormalizowanych i zdenormalizowanych w formatach arytymetyki `Float32` oraz `Float64`.

```
In [10]: using PrettyTables

min_sub_32 = nextfloat(Float32(0))
min_sub_64 = nextfloat(Float64(0))
min_nor_32 = floatmin(Float32)
min_nor_64 = floatmin(Float64)

data = ["Float32" "$min_sub_32" "$min_nor_32";
        "Float64" "$min_sub_64" "$min_nor_64"]

pretty_table(data, header=["Type", "MIN_sub", "MIN_nor"], alignment=:l, h
```

Type	MIN_sub	MIN_nor
Float32	1.0e-45	1.1754944e-38
Float64	5.0e-324	2.2250738585072014e-308

Wprowadzenie liczb zdenormalizowanych umożliwia uzyskanie wartości, które są nawet o 7 rzędów mniejsze w arytymetyce `Float32` oraz o 16 rzędów mniejsze w arytymetyce `Float64` w porównaniu do najmniejszych liczb w tych formatach.

Nieskończoność w standardzie IEEE 754

Większość nowoczesnych systemów komputerowych wykorzystuje standard IEEE 754 do reprezentacji liczb zmiennoprzecinkowych. W ramach tego standardu, nieskończoność jest reprezentowana jako specjalna wartość.

W arytymetyce zmiennoprzecinkowej nieskończoność może być dodatnia ($+\infty$) lub ujemna ($-\infty$). Dodatnia nieskończoność jest używana w przypadku przekroczenia górnej granicy reprezentacji liczb, a ujemna nieskończoność w przypadku dolnej granicy.

Gdy wykonuje się operacje arytymetyczne z nieskończonością, istnieją określone zasady dotyczące wyników:

- $x + \infty = \infty$ dla każdej liczby x .
- $x - \infty = -\infty$ dla każdej liczby x .
- $\infty \cdot x = \infty$ dla $x > 0$ oraz $-\infty$ dla $x < 0$.
- $\frac{\infty}{\infty}$ oraz $\frac{-\infty}{-\infty}$ są nieokreślone, co może prowadzić do trudności w obliczeniach.

W standardzie IEEE 754 nieskończoność jest reprezentowana w następujący sposób:

- **Bit znaku:** określa, czy jest to dodatnia ($+\infty$) czy ujemna ($-\infty$) nieskończoność
- **Cecha:** wszystkie bity są ustawione na jedynki
- **Mantysa:** wszystkie bity są ustawione na zera

Największa liczba możliwa do zapisania w standardzie IEEE 754 ma następującą strukturę:

- **Bit znaku:** ustawiony na 0 (oznaczający liczbę dodatnią)
- **Cecha (eksponent):** najmniej znaczący bit ustawiony na 0, reszta bitów ustawiona na 1
- **Mantysa:** wszystkie bity ustawione na jedynki

Eksperymentalne wyznaczenie maksymalnych wartości w arytmetykach zmiennoprzecinkowych

Algorytm do wyznaczania maksymalnej wartości w arytmetykach zmiennoprzecinkowych rozpoczyna się od liczby, która ma mantysę wypełnioną samymi jedynkami oraz cechą równą *bias* — 1. Ta struktura liczby oznacza, że znajduje się ona tuż poniżej 1, która ma mantysę wypełnioną samymi zerami, a cechą równą *bias*. Poprzez mnożenie tej liczby przez 2 będziemy stopniowo zwiększać cechę, aż do momentu, gdy nie będzie ona wypełniona samymi jedynkami (do momentu aż nie osiągniemy ∞). Maksymalną liczbą będzie zatem ostatnia liczba, którą napotkamy przed osiągnięciem nieskończoności.

Poprawność algorytmu zapewnia, więc warunek `while !isinf(type(2) * max)`, gdzie `max` jest liczbą, którą mnożymy przez 2.

```
In [39]: function compute_max(type)
           max = prevfloat(type(1))
           while !isinf(type(2) * max)
               max *= type(2)
           end
           return max
       end
```

```
Out[39]: compute_max (generic function with 1 method)
```

W celu dodatkowego sprawdzenia poprawności algorytmu, uzyskane wyniki zostały zweryfikowane z wartościami zwróconymi przez funkcję `floatmax()` z języka Julia oraz z danymi zawartymi w pliku nagłówkowym `<float.h>` jako `FLT_MAX` dla `Float32` oraz `DBL_MAX` dla `Float64`.

```
In [41]: max_16 = compute_max(Float16)
max_32 = compute_max(Float32)
max_64 = compute_max(Float64)

data = ["Float16" "$max_16" "$(floatmax(Float16))" "-";
        "Float32" "$max_32" "$(floatmax(Float32))" "3.4028235e38";
        "Float64" "$max_64" "$(floatmax(Float64))" "1.7976931348623157e308"]

pretty_table(data, header=["Type", "Max", "Float max (Julia)", "Float max (C)"])
```

Type	Max	Float max (Julia)	Float max (C)
Float16	6.55e4	6.55e4	-
Float32	3.4028235e38	3.4028235e38	3.4028235e38
Float64	1.7976931348623157e308	1.7976931348623157e308	1.7976931348623157e308

Wnioski

Wyniki uzyskane za pomocą algorytmu `compute_max` dla typów `Float16`, `Float32` oraz `Float64` są zgodne z wartościami zwróconymi przez funkcję `floatmax()` w języku Julia oraz z wartościami `FLT_MAX` i `DBL_MAX` z pliku nagłówkowego `<float.h>` w języku C. To wskazuje na poprawność implementacji algorytmu oraz jego zgodność ze standardem IEEE 754.

Widoczne są duże różnice w zakresie pomiędzy arytmetykami `Float16`, `Float32` oraz `Float64`. To pokazuje, że mniejsze typy danych mają ograniczoną precyzję oraz zakres.

Gęstość rozmieszczenia liczb w arytmetykach zmiennoprzecinkowych

W obszarze arytmetyki zmiennoprzecinkowej, standard IEEE 754, szczególnie w wersji `Float64`, odgrywa kluczową rolę w definiowaniu sposobu reprezentacji liczb. Liczby zmiennoprzecinkowe w tym standardzie charakteryzują się równomiernym rozmieszczeniem w określonych przedziałach, co ma istotne znaczenie dla precyzji obliczeń numerycznych.

W przedziale $[1, 2]$, liczby te można opisać za pomocą wzoru:

$$x = 1 + k\delta \quad \text{dla} \quad k = 0, 1, 2, \dots, 2^{52} - 1$$

gdzie $\delta = 2^{-52}$ jest epsilonem maszynowym dla arytmetyki `Float64`. Oznacza to, że liczby zmiennoprzecinkowe w tym zakresie są rozmieszczone z krokiem równym δ , co zapewnia ich precyzyjną reprezentację.

Eksperymentalne wyznaczanie wartości k

Aby potwierdzić powyższe założenia dotyczące rozmieszczenia liczb zmiennoprzecinkowych w arytmetyce `Float64`, konieczne jest opracowanie metody wyznaczania wartości k dla dowolnej liczby x z przedziału $[1, 2]$. Matematycznie, wartość k można wyrazić za pomocą następującego wzoru:

$$k = \frac{x - 1}{\delta}$$

gdzie $\delta = 2^{-52}$ reprezentuje krok, w jakim rozmieszczone są liczby zmiennoprzecinkowe.

Zgodnie z założeniem, jeżeli liczby w arytmetyce `Float64` są równomiernie rozmieszczone w przedziale $[1, 2]$, to wartość k odpowiada mantysie liczby x .

Wykorzystując te dwa podejścia, można zatem zaimplementować algorytm, który będzie testował równomierność rozmieszczenia liczb `Float64` w analizowanym przedziale. Algorytm ten pozwoli na weryfikację, czy wszystkie liczby w tym zakresie są zgodne z przewidywanym rozkładem.

Jako, że sprawdzenie wszystkich liczb z przedziału $[1, 2]$ jest operacją kosztowną, do eksperymentu wybierane jest 10^6 liczb z podanego przedziału, zgodnie z rozkładem jednostajnym.

```
In [50]: using Distributions

function check_distribution(start, stop, delta)

    success = 0
    error = 0

    for i in 1:1000000

        x = rand(Uniform(start, stop))
        bitstring_x = bitstring(x)
        last_52_digits = bitstring_x[end-51:end]
        integer_value = parse{Int, last_52_digits; base=2}
        x_recounted = start + integer_value * delta

        if (x != x_recounted)
            error += 1
        else
            success += 1
        end
    end

    return success, error
```

```
end
```

```
Out[50]: check_distribution (generic function with 1 method)
```

```
In [49]: using PrettyTables
```

```
success, error = check_distribution(1, 2, 2^(-52))
```

```
data = ["1" "2" "2^(-52)" "$success" "$error"]
```

```
pretty_table(data, header=["Start", "Stop", "Delta", "Success", "Error"],
```

Start	Stop	Delta	Success	Error
1	2	2 ⁽⁻⁵²⁾	100000	0

W przeprowadzonym eksperymencie udało się znaleźć prawidłową wartość k dla wszystkich wylosowanych x , co pozwala na potwierdzenie równomiernego rozkładu liczb `Float64` z krokiem $\delta = 2^{-52}$ w przedziale $[1, 2]$.

Gęstość rozmieszczenia liczb w przedziale $[0.5, 1]$

W przedziale $[0.5, 1]$ liczby mają postać $(0, 1 \dots)_2$. Oznacza to, że na mantysie jesteśmy w stanie określić cyfry w przedstawieniu do 53 miejsca po przecinku, a nie 52, jak w przypadku liczb z przedziału $[1, 2]$. Zwiększa nam to precyzję, a także gęstość rozmieszczenia liczb.

W tym przypadku $\delta = 2^{-53}$, a liczby można przedstawić jako:

$$x = 1 + k\delta \quad \text{dla} \quad k = 0, 1, 2, \dots, 2^{53} - 1$$

Hipotezę tę również potwierdzimy eksperymentem opartym na algorytmie zaprezentowanym powyżej.

```
In [52]: using PrettyTables
```

```
success, error = check_distribution(0.5, 1, 2^(-53))
```

```
data = ["1" "2" "2^(-53)" "$success" "$error"]
```

```
pretty_table(data, header=["Start", "Stop", "Delta", "Success", "Error"],
```

Start	Stop	Delta	Success	Error
1	2	2 ⁽⁻⁵³⁾	100000	0

Jak widać w przedstawionych wynikach udało się znaleźć prawidłową wartość k dla wszystkich wylosowanych x , co pozwala na potwierdzenie równomiernego rozkładu liczb `Float64` z krokiem $\delta = 2^{-53}$ w przedziale $[0.5, 1]$.

Gęstość rozmieszczenia liczb w przedziale $[2, 4]$

W przedziale $[2, 4]$ sytuacja jest odwrotna. Ze względu na występowanie dwóch znaczących liczb przed przecinkiem w reprezentacji dwójkowej, tj. $(10, \dots)_2$ lub $(11, \dots)_2$ możemy określać liczby po przecinku jedynie do 51 miejsca. Zmniejsza nam to precyzję oraz gęstość rozmieszczenia liczb.

W tym przypadku $\delta = 2^{-51}$, a liczby można przedstawić jako:

$$x = 1 + k\delta \quad \text{dla} \quad k = 0, 1, 2, \dots, 2^{51} - 1$$

.

Hipotezę tę również potwierdzimy eksperymentem opartym na tym samym algorytmie.

```
In [54]: using PrettyTables

success, error = check_distribution(2, 4, 2^(-51))

data = ["1" "2" "2^(-53)" "$success" "$error"]

pretty_table(data, header=["Start", "Stop", "Delta", "Success", "Error"],
```

Start	Stop	Delta	Success	Error
1	2	2 ⁽⁻⁵³⁾	100000	0

Jak widać w przedstawionych wynikach udało się znaleźć prawidłową wartość k dla wszystkich wylosowanych x , co pozwala na potwierdzenie równomiernego rozkładu liczb `Float64` z krokiem $\delta = 2^{-51}$ w przedziale $[2, 4]$.

Przypadek ogólny

W ogólności gęstość rozmieszczenia liczb zmiennoprzecinkowych (δ) w przedziale $[2^a, 2^b]$, gdzie a i b są kolejnymi potęgami 2, wynosi $2^{-(52-a)}$.

Gęstość ta jest wyznaczana przez liczbę cyfr w reprezentacji binarnej liczby przed przecinkiem.

Analiza błędów obliczeniowych

Ze względu na ograniczoną liczbę bitów, które możemy wykorzystać do reprezentacji liczb w pamięci komputera, często dochodzi do błędów obliczeniowych wynikających z operacji na przybliżonych wartościach zamiast na ich rzeczywistych reprezentacjach. W tej sekcji przedstawione zostaną cztery przykłady eksperymentów, które ujawniają niedokładności obliczeń w systemach komputerowych.

$$x \cdot \frac{1}{x} \neq 1$$

Na pierwszy rzut oka wydaje się, że wyrażenie $x \cdot \frac{1}{x}$ powinno zawsze być równe 1, pod warunkiem, że x jest różne od zera. W praktyce, jednak z powodu ograniczonej precyzji arytmetyki zmiennoprzecinkowej w komputerach, mogą wystąpić sytuacje, w których wynik tej operacji nie będzie równy 1. Problem ten jest szczególnie widoczny w przypadku wartości bliskich zera lub bardzo dużych liczb.

W celu ukazania problemu zaimplementowany został algorytm, który przechodzi po kolejnych liczbach w arytmetyce `Float64` zaczynając od 0 i szuka takiej liczby, dla której nierówność $x \cdot \frac{1}{x} \neq 1$ jest spełniona.

```
In [13]: function find_smallest_weird_nmb()
          nmb = 1.0
          while nmb * (1.0 / nmb) == 1.0
              nmb = nextfloat(nmb)
          end

          return nmb
        end
```

```
Out[13]: find_smallest_weird_nmb (generic function with 1 method)
```

```
In [17]: using PrettyTables

          val = find_smallest_weird_nmb()

          data = ["x" "$val" "$(bitstring(val))";
                  "1 / x" "$(1 / val)" "$(bitstring((1 / val)))";
                  "x * 1/x" "$(val * (1 / val))" "$(bitstring(val * (1 / val)))";
                  "x * 1/x + 2^(-52)" "$((val * (1 / val)) + 2^(-52))" "$(bitstring(
pretty_table(data, header=["Name", "Value", "Bitstring"], alignment=:l, h
```

Name	Value	Bitstring
x	1.0000000057228997	00111111111100000000000000000000
1 / x	0.9999999427710061	00111111111011111111111111111111
x * 1/x	0.9999999999999999	00111111111011111111111111111111
x * 1/x + 2 ⁽⁻⁵²⁾	1.0	00111111111100000000000000000000

W wyniku przeprowadzonych eksperymentów ustalono, że najmniejsza liczba, która spełnia nierówność $x \cdot \frac{1}{x}$, wynosi 1.0000000057228997. Mnożąc tę liczbę przez jej odwrotność, uzyskujemy wynik równy 0.9999999999999999. W wyniku operacji dzielenia i mnożenia otrzymujemy błąd równy 2^{-52} .

Iloczyn skalarny

Iloczyn skalarny dwóch wektorów w przestrzeni euklidesowej to operacja, która łączy dwa wektory i zwraca liczbę (skalar). Definicja iloczynu skalarnego wektorów $\mathbf{a} = (a_1, a_2, \dots, a_n)$ oraz $\mathbf{b} = (b_1, b_2, \dots, b_n)$ jest następująca:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i$$

Ze względu na dużą liczbę mnożeń i dodawań w operacji tworzenia iloczynu skalarnego, dochodzi do względnie dużych błędów obliczeniowych.

W celu ukazania tych błędów zaimplementowane zostały cztery alternatywne metody wyprowadzenia iloczynu skalarnego, a następnie wszystkie wyniki skonfrontowano z prawdziwym wartością iloczynu skalarnego.

Sumowanie w przód

Pierwszą zastosowaną metodą jest policzenie iloczynu skalarnego zgodnie z podanym wyżej wzorem, tj. sumowanie iloczynów kolejnych składowych wektora.

```
In [18]: function scalar_forward(a, b, type)
    sum = type(0)
    for i in 1:length(a)
        sum += type(type(a[i]) * type(b[i]))
    end

    return sum
end
```

Out[18]: scalar_forward (generic function with 1 method)

Sumowanie w tył

Drugim podejściem było odwrócenie wzoru:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=n}^1 a_i b_i$$

```
In [19]: function scalar_backwards(a, b, type)
          sum = type(0)
          for i in length(a):-1:1
              sum += type(type(a[i]) * type(b[i]))
          end

          return sum
        end
```

Out[19]: scalar_backwards (generic function with 1 method)

Sumowanie od największego do najmniejszego

Kolejną alternatywną metodą jest początkowe zebranie wszystkich iloczynów, a następnie dodanie ich od największej wartości do najmniejszej (w przypadku dodatnich iloczynów) albo najmniejszej do największej (w przypadku ujemnych iloczynów).

```
In [20]: function scalar_max_min(a, b, type)
          pos = []
          neg = []
          for i in 1:length(a)
              c = type(type(a[i]) * type(b[i]))

              if c > 0
                  push!(pos, c)
              else
                  push!(neg, c)
              end
          end

          pos = sort(pos, rev=true)
          neg = sort(neg)

          sum_pos = type(0)
          for i in 1:length(pos)
              sum_pos += type(pos[i])
          end

          sum_neg = type(0)
          for i in 1:length(neg)
              sum_neg += type(neg[i])
          end

          return type(type(sum_pos) + type(sum_neg))
        end
```

Out[20]: scalar_max_min (generic function with 1 method)

Sumowanie od najmniejszego do największego

Ostatnie użyte podejście jest niemal identyczne jak sumowanie od największego do najmniejszego. Została jednak zastosowana odwrotna kolejność sumowania iloczynów dodatnich i ujemnych.

```
In [21]: function scalar_min_max(a, b, type)
    pos = []
    neg = []
    for i in 1:length(a)
        c = type(type(a[i]) * type(b[i]))

        if c > 0
            push!(pos, c)
        else
            push!(neg, c)
        end
    end

    pos = sort(pos)
    neg = sort(neg, rev=true)

    sum_pos = type(0)
    for i in 1:length(pos)
        sum_pos += type(pos[i])
    end

    sum_neg = type(0)
    for i in 1:length(neg)
        sum_neg += type(neg[i])
    end

    return type(type(sum_pos) + type(sum_neg))
end
```

Out[21]: scalar_min_max (generic function with 1 method)

Wektorami wybranymi do testowania wyżej wymienionych metod były:

$x = [2.718281828, -3.141592654, 1.414213562, 0.5772156649, 0.3010299957]$

$y = [1486.2497, 878366.9879, -22.37492, 4773714.647, 0.000185049]$

Rzeczywistą wartością iloczynu skalarnego wektorów x i y jest $-1.00657107000000 \cdot 10^{-11}$.

```
In [28]: using PrettyTables

a = [2.718281828, -3.141592654, 1.414213562, 0.5772156649, 0.3010299957]
b = [1486.2497, 878366.9879, -22.37492, 4773714.647, 0.000185049]
real = -1.00657107000000 * 10^(-11)

data = ["Float32" "$(scalar_forward(a, b, Float32))" "$(scalar_backwards(
    "Float64" "$(scalar_forward(a, b, Float64))" "$(scalar_backwards(
```

```
"Float64 (real)" "$real" "$real" "$real" "$real"]
```

```
pretty_table(data, header=["Type", "Forward", "Backwards", "Max -> Min",
```

Type -> Min	Forward Min -> Max	Backwards	Max
Float32	-0.4999443	-0.4543457	-0.5
Float64	1.0251881368296672e-10	-1.5643308870494366e-10	0.0
Float64 (real)	-1.0065710699999998e-11	-1.0065710699999998e-11	-1.0065710699999998e-11

Przykład ten został również przetestowany za pomocą funkcji `dot()` w języku Julia, funkcji `dot product{}` w języku Wolfram Alpha oraz kalkulatorów w systemach operacyjnych Linux i Android. Wyniki tych eksperymentów prezentują się następująco:

```
In [34]: using PrettyTables

data = ["Julia" "1.0251881368296672e-10";
        "Wolfram Alpha" "1.0251881368296672e-10";
        "Linux" "0";
        "Android" "-1.0065710699999998e-11"]

pretty_table(data, header=["Platform", "Result"], alignment=:l, header_cr
```

Platform	Result
Julia	1.0251881368296672e-10
Wolfram Alpha	1.0251881368296672e-10
Linux	0
Android	-1.0065710699999998e-11

Analiza wyników przeprowadzonych eksperymentów

Wyniki eksperymentu przeprowadzonego na wektorach x i y uwydatniają kluczowe znaczenie kolejności operacji matematycznych w kontekście obliczeń numerycznych. W przypadku małych wartości, jak w Float64, efekty zaokrągleń mogą prowadzić do znaczących odchyłeń, co można zauważyć w różnicach między poszczególnymi metodami obliczeniowymi.

Różnice w wynikach między algorytmami i platformami podkreślają, jak różne systemy mogą interpretować operacje na liczbach zmiennoprzecinkowych.

Analiza wyników uzyskanych w języku Julia i na platformie Wolfram Alpha wykazuje spójność, co może sugerować, że obie platformy stosują zbliżone algorytmy do obliczeń numerycznych. Wyniki uzyskane metodą "Forward" z wykorzystaniem

arytmetyki `Float64` są zbliżone do tych na wymienionych platformach, co może wskazywać na podobieństwo zastosowanych algorytmów i precyzji obliczeń.

Zauważalna jest również zbieżność wyników w metodach `Min -> Max` i `Max -> Min` w arytmetyce `Float64` oraz na systemie Linux. To może sugerować, że użyty algorytm i implementacja arytmetyki w tym systemie są podobne do tych zastosowanych w eksperymentalnych metodach.

W kontekście metod iteracyjnych, takich jak dodawanie czy mnożenie, błąd zaokrągleń ma tendencję do kumulacji, co może prowadzić do istotnych różnic w wynikach. Zmiana kolejności operacji wpływa na sposób kumulacji tych błędów. Dlatego obliczenia numeryczne powinny być przeprowadzane w sposób minimalizujący błędy zaokrągleń, co często wymaga przemyślenia kolejności działań.

Czy $f = g$?

Trzecim z eksperymentów, który ukazuje istotę błędów zaokrągleń w arytmetykach zmiennoprzecinkowych było wyliczenie wartości funkcji na dwa sposoby. Dane były funkcje:

$$f(x) = \sqrt{x^2 + 1} - 1$$

$$g(x) = \frac{x^2}{\sqrt{x^2 + 1} + 1}$$

Aby wykazać, że funkcje $f(x)$ i $g(x)$ są sobie równe, możemy przekształcić jedną z funkcji w drugą. Spróbujmy przekształcić $f(x)$ w $g(x)$ lub odwrotnie, aby uzyskać równoważne wyrażenie.

Rozpocznijmy od funkcji:

$$f(x) = \sqrt{x^2 + 1} - 1$$

Aby uprościć to wyrażenie, przemnożymy je przez wyrażenie sprzężone $(\sqrt{x^2 + 1} + 1)$ (zarówno licznik, jak i mianownik):

$$f(x) = (\sqrt{x^2 + 1} - 1) \cdot \frac{\sqrt{x^2 + 1} + 1}{\sqrt{x^2 + 1} + 1}$$

Po wykonaniu mnożenia w liczniku otrzymujemy:

$$f(x) = \frac{(\sqrt{x^2 + 1})^2 - 1^2}{\sqrt{x^2 + 1} + 1}$$

Rozwijamy teraz wyrażenia w liczniku:

$$f(x) = \frac{x^2 + 1 - 1}{\sqrt{x^2 + 1} + 1}$$

Uprościmy licznik:

$$f(x) = \frac{x^2}{\sqrt{x^2 + 1} + 1}$$

Zatem:

$$f(x) = g(x)$$

Wykazaliśmy, że $f(x)$ i $g(x)$ są sobie równe, czyli:

$$f(x) = g(x)$$

W celu weryfikacji tej równości zaimplementowano algorytm, który sprawdza, czy funkcje $f(x)$ i $g(x)$ są faktycznie obliczane w ten sam sposób w pamięci komputera. Algorytm ten porównuje wartości obu funkcji dla wartości 8^{-i} , gdzie $i \in \{1, \dots, 13\}$, analizując potencjalne różnice wynikające z zaokrągleń lub kolejności operacji w arytmetyce zmiennoprzecinkowej.

Takie podejście pozwala na ocenę, czy równość tych funkcji teoretyczna znajduje potwierdzenie także w praktyce obliczeń numerycznych.

```
In [35]: using PrettyTables

f(x) = sqrt(x^2 + 1.0) - 1.0
g(x) = x^2 / (sqrt(x^2 + 1.0) + 1.0)

function create_table()
    data = []

    for i in 1:13
        x = 8.0^(-i)
        f_x = f(x)
        g_x = g(x)
        x_sqrt = sqrt(x^2 + 1.0)
        if (data == [])
            data = ["$i" "$x" "$x_sqrt" "$f_x" "$g_x"]
        else
            new_row = ["$i" "$x" "$x_sqrt" "$f_x" "$g_x"]

            data = vcat(data, new_row)
        end
    end

    return data
end

results = create_table()

pretty_table(results, header=["i", "x", "sqrt(x**2 + 1)", "f(x)", "g(x)"])
```

i		x	$\sqrt{x^2 + 1}$	f(x)
		g(x)		
3	1	0.125	1.0077822185373186	0.0077822185373186414
	2	0.015625	1.0001220628628287	0.0001220628628286757
	3	0.00012206286282875901	1.0000019073468138	1.9073468138230965e-6
3	4	0.001953125	1.000000029802322	2.9802321943606103e-8
	5	0.000244140625	1.0000000004656613	4.656612873077393e-10
	6	3.0517578125e-5	1.000000000007276	7.275957614183426e-12
3	7	3.814697265625e-6	1.0000000000001137	1.1368683772161603e-1
	8	4.76837158203125e-7	1.0000000000000018	1.7763568394002505e-1
	9	5.960464477539063e-8	1.0	0.0
5	10	7.450580596923828e-9	1.0	0.0
	11	2.7755575615628914e-17	1.0	0.0
	12	9.313225746154785e-10	1.0	0.0
5	13	4.336808689942018e-19	1.0	0.0
	14	1.1641532182693481e-10	1.0	0.0
	15	6.776263578034403e-21	1.0	0.0
5	16	1.4551915228366852e-11	1.0	0.0
	17	1.0587911840678754e-22	1.0	0.0
	18	1.8189894035458565e-12	1.0	0.0
5	19	1.6543612251060553e-24		
	20			
	21			

Jak się okazuje, funkcje w pamięci komputera są liczone inaczej. Funkcja $f(x)$ od pewnego momentu przyjmuje wartość 0. Wynika to z faktu, że pierwiastek w tej funkcji, z powodu błędów obliczeniowych, wynosi 1.

Wiarygodność wyników

Dla:

- $i \in \{1, \dots, 8\}$: wartości obu funkcji są dodatnie, niezerowe i dążą do zera, co może wskazywać na wiarygodność wyników. Funkcje zachowują się zgodnie z oczekiwaniami, a różnice między ich wartościami są minimalne.
- $i \in \{9, \dots, 13\}$: wartości funkcji $f(x)$ są równe zero, co wskazuje na brak wiarygodności wyników. Błąd zaokrąglenia spowodował, że pierwiastek w tych przypadkach został obliczony jako 1, prowadząc do wyniku zero. Z kolei wartości funkcji $g(x)$ nadal zbliżają się do zera, ale pozostają dodatnie i niezerowe, co sugeruje, że obliczenia w tym zakresie mogą być wiarygodne, mimo że są one bliskie granicy precyzji arytmetyki zmiennoprzecinkowej.

Przybliżona wartość pochodnej vs prawdziwa wartość

Przybliżoną wartość pochodnej $f'(x)$ w punkcie x_0 można obliczyć za pomocą następującego wzoru:

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}$$

Aby zbadać błąd związany z obliczaniem pochodnej według wzoru aproksymacyjnego zaimplementowano algorytm, który analizuje różnicę między wzorem przybliżającym wartość oraz faktycznym wzorem na pochodną dla funkcji $f(x) = \sin(x) + \cos(3x)$ w punkcie $x_0 = 1$ oraz dla $h = 2^{-n}$ ($n \in \{0, 1, 2, \dots, 54\}$).

```
In [37]: function derivative(f, x, h)
          return (f(x + h) - f(x)) / h
        end
```

Out[37]: derivative (generic function with 1 method)

```
In [38]: f(x) = sin(x) + cos(3 * x)
          df(x) = cos(x) - 3 * sin(3 * x)

          x = 1.0

          function create_table()
              data = []

              for i in 0:54
                  x = 1.0
                  h = 2.0^(-i)
                  f_x = derivative(f, x, h)
                  f_x_df_x = abs(f_x - df(x))
                  h_1 = 1.0 + h
                  if (data == [])
                      data = ["$i" "$h" "$h_1" "$f_x" "$f_x_df_x"]
                  else
                      new_row = ["$i" "$h" "$h_1" "$f_x" "$f_x_df_x"]

                      data = vcat(data, new_row)
                  end
              end

              return data
          end

          results = create_table()

          pretty_table(results, header=["i", "h", "1+h", "f'(x)", "|f'(x) - f'_tild
```

i		h	1+h	f'(x)
f'(x) - f'_tilde_x				
0	1.0	2.0	2.0179892252685967	
1	0.5	1.5	1.8704413979316472	
2	0.25	1.25	1.1077870952342974	
3	0.125	1.125	0.6232412792975817	
4	0.0625	1.0625	0.3704000662035192	
5	0.03125	1.03125	0.24344307439754687	
6	0.015625	1.015625	0.18009756330732785	
7	0.0078125	1.0078125	0.1484913953710958	
8	0.00390625	1.00390625	0.1327091142805159	
9	0.001953125	1.001953125	0.1248236929407085	
10	0.0009765625	1.0009765625	0.12088247681106168	
11	0.00048828125	1.00048828125	0.11891225046883847	
12	0.000244140625	1.000244140625	0.11792723373901026	
13	0.0001220703125	1.0001220703125	0.11743474961076572	
14	6.103515625e-5	1.00006103515625	0.11718851362093119	
15	3.0517578125e-5	1.000030517578125	0.11706539714577957	
16	1.52587890625e-5	1.0000152587890625	0.11700383928837255	
17	7.62939453125e-6	1.0000076293945312	0.11697306045971345	
18	3.814697265625e-6	1.0000038146972656	0.11695767106721178	
19	1.9073486328125e-6	1.0000019073486328	0.11694997636368498	
20	9.5367431640625e-7	1.0000009536743164	0.11694612901192158	
21	4.76837158203125e-7	1.0000004768371582	0.1169442052487284	
22	2.384185791015625e-7	1.000000238418579	0.11694324295967817	
23	1.1920928955078125e-7	1.0000001192092896	0.11694276239722967	
24	5.960464477539063e-8	1.0000000596046448	0.11694252118468285	
25	2.9802322387695312e-8	1.0000000298023224	0.116942398250103	
26	1.4901161193847656e-8	1.0000000149011612	0.11694233864545822	

27	7.450580596923828e-9	1.00000000074505806	0.11694231629371643	
3.460517827846843e-8				
28	3.725290298461914e-9	1.00000000037252903	0.11694228649139404	
4.802855890773117e-9				
29	1.862645149230957e-9	1.00000000018626451	0.11694222688674927	
5.480178888461751e-8				
30	9.313225746154785e-10	1.00000000009313226	0.11694216728210449	
1.1440643366000813e-7				
31	4.656612873077393e-10	1.00000000004656613	0.11694216728210449	
1.1440643366000813e-7				
32	2.3283064365386963e-10	1.00000000002328306	0.11694192886352539	
3.5282501276157063e-7				
33	1.1641532182693481e-10	1.00000000001164153	0.11694145202636719	
8.296621709646956e-7				
34	5.820766091346741e-11	1.00000000000582077	0.11694145202636719	
8.296621709646956e-7				
35	2.9103830456733704e-11	1.00000000000291038	0.11693954467773438	
2.7370108037771956e-6				
36	1.4551915228366852e-11	1.0000000000014552	0.116943359375	
1.0776864618478044e-6				
37	7.275957614183426e-12	1.0000000000007276	0.1169281005859375	
1.4181102600652196e-5				
38	3.637978807091713e-12	1.0000000000003638	0.116943359375	
1.0776864618478044e-6				
39	1.8189894035458565e-12	1.0000000000001819	0.11688232421875	
5.9957469788152196e-5				
40	9.094947017729282e-13	1.00000000000009095	0.1168212890625	
0.0001209926260381522				
41	4.547473508864641e-13	1.00000000000004547	0.116943359375	
1.0776864618478044e-6				
42	2.2737367544323206e-13	1.00000000000002274	0.11669921875	
0.0002430629385381522				
43	1.1368683772161603e-13	1.00000000000001137	0.1162109375	
0.0007313441885381522				
44	5.684341886080802e-14	1.00000000000000568	0.1171875	
0.0002452183114618478				
45	2.842170943040401e-14	1.00000000000000284	0.11328125	
0.003661031688538152				
46	1.4210854715202004e-14	1.00000000000000142	0.109375	
0.007567281688538152				
47	7.105427357601002e-15	1.0000000000000007	0.109375	
0.007567281688538152				
48	3.552713678800501e-15	1.00000000000000036	0.09375	
0.023192281688538152				
49	1.7763568394002505e-15	1.00000000000000018	0.125	
0.008057718311461848				
50	8.881784197001252e-16	1.00000000000000009	0.0	
0.11694228168853815				
51	4.440892098500626e-16	1.00000000000000004	0.0	
0.11694228168853815				
52	2.220446049250313e-16	1.00000000000000002	-0.5	
0.6169422816885382				
53	1.1102230246251565e-16	1.0	0.0	
0.11694228168853815				
54	5.551115123125783e-17	1.0	0.0	
0.11694228168853815				

Analiza uzyskanych wyników

Gdy zmniejszamy wartość h , zyskujemy większą precyzję w obliczeniach. Jednak zmniejszanie h do zbyt małych wartości prowadzi do dominacji błędów zaokrągleń oraz kumulacji błędów obliczeniowych. Wartości $f(x_0 + h)$ i $f(x_0)$ stają się zbyt bliskie sobie, a w konsekwencji różnica między nimi, która ma być używana w obliczeniach, staje się porównywalna z błędem numerycznym. To prowadzi do sytuacji, w której obliczany przyrost $f'(x_0)$ może być mniej dokładny niż przy większym h .

W rezultacie, chociaż teoretycznie mniejsze h powinno poprawiać dokładność przybliżenia pochodnej, w praktyce może to prowadzić do coraz większych odchyłeń od rzeczywistej wartości pochodnej.

Zwykle dla każdego problemu istnieje punkt optymalny dla h , przy którym błąd numeryczny jest minimalny. Po przekroczeniu tej wartości, dalsze zmniejszanie h nie poprawia, a wręcz pogarsza jakość wyników. W naszym przypadku jest to około $h = 10$.

Wnioski

- **Ograniczenia arytmetyki zmiennoprzecinkowej**

Wyniki eksperymentów pokazują, że ograniczona precyzja arytmetyki zmiennoprzecinkowej prowadzi do nieoczekiwanych błędów obliczeniowych, nawet w prostych operacjach, takich jak $x \cdot \frac{1}{x}$. Podkreśla to konieczność stosowania odpowiednich algorytmów numerycznych w kontekście obliczeń o wysokiej precyzji.

- **Kumulacja błędów w operacjach matematycznych**

Eksperymenty związane z iloczynem skalarnym ujawniają, że kumulacja błędów zaokrągleń staje się poważnym problemem w przypadku operacji z dużą liczbą mnożeń i dodawań. Wartości uzyskiwane w różnych metodach obliczeniowych mogą się znacznie różnić, co wskazuje na to, że kolejność operacji ma kluczowe znaczenie.

- **Praktyczna równość funkcji a teoretyczne przekształcenia**

Analiza funkcji $f(x)$ i $g(x)$ pokazuje, że chociaż matematycznie można wykazać ich równość, w praktyce obliczenia numeryczne mogą prowadzić do różnych wyników z powodu błędów zaokrągleń.

- **Optymalizacja wartości kroków w przybliżonej obliczaniu pochodnych**

Zmniejszanie wartości h w przybliżeniu pochodnej prowadzi do zyskania większej precyzji, ale także do wzrostu błędów numerycznych, gdy h staje się zbyt małe. W każdym przypadku istnieje punkt optymalny, przy którym błąd numeryczny jest minimalny, co wskazuje, że dobór odpowiedniej wartości h jest kluczowy dla uzyskania wiarygodnych wyników.